Bioinformatics Pipeline

Install & import packages

```
import google.colab.files as files
from pathlib import Path
```

Input or fetch data

```
# Upload a FASTA file(s)
uploaded = files.upload()

# Process the uploaded files
for filename in uploaded.keys():
    print(f'Uploaded file: {filename}')
    # You can now read and process the file content, for example:
    # with open(Path(filename).resolve(), 'r') as f:
    #     fasta_content = f.read()
    #     print(f"Content of {filename}:\n{fasta_content[:200]}...") # Print first 200 chars
```

Choose Files Mnemiopsi…subset.fasta

Mnemiopsis_cathepsin_subset.fasta(n/a) - 4580 bytes, last modified: 4/8/2026 - 100% done

```
Saving Mnemiopsis_cathepsin_subset.fasta to Mnemiopsis_cathepsin_subset (2).fasta
Uploaded file: Mnemiopsis_cathepsin_subset (2).fasta
```

Store FASTA File

```
# Extract the filename(s) from the uploaded dictionary
fasta_files = list(uploaded.keys())

fasta_data_dict = {}

if fasta_files:
    print(f"Found {len(fasta_files)} FASTA file(s) for analysis.")

    # Read the content of all files into a dictionary
    for file_path in fasta_files:
        with open(file_path, 'r') as f:
            fasta_data_dict[file_path] = f.read()
        print(f"Successfully loaded {len(fasta_data_dict[file_path])} characters from {file_path}.")
else:
    print("No files were uploaded in the previous step. Please run the upload cell first.")
```

```
Found 1 FASTA file(s) for analysis.
Successfully loaded 4580 characters from Mnemiopsis_cathepsin_subset (2).fasta.
```

Calculate Psychochemical Properties

```
!pip install biopython
```

```
Requirement already satisfied: biopython in /usr/local/lib/python3.12/dist-packages (1.87)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from biopython) (2.0.2)
```

```
from Bio import SeqIO
```

```
from Bio.SeqUtils.ProtParam import ProteinAnalysis
import io

if 'fasta_data_dict' in locals() and fasta_data_dict:
    for filename, fasta_data in fasta_data_dict.items():
        print(f"\n{'='*50}\nProcessing File: {filename}\n{'='*50}")
        # Read the FASTA data from the stored variable
        records = list(SeqIO.parse(io.StringIO(fasta_data), 'fasta'))

        for record in records:
            print(f"\n--- Physiochemical Properties for: {record.id} ---")

            # Convert sequence to string and remove stop codons if present
            sequence = str(record.seq).replace('*', '')

            # Initialize ProteinAnalysis object
            analyzed_seq = ProteinAnalysis(sequence)

            # 1. Amino acid length
            print(f"Length: {len(sequence)} amino acids")

            # 2. Molecular weight
            mw = analyzed_seq.molecular_weight()
            print(f"Molecular Weight: {mw:.2f} Da")

            # 3. Amino acid composition
            aa_comp = analyzed_seq.count_amino_acids()
            print("Amino Acid Composition:")
            for aa, count in aa_comp.items():
                print(f"  {aa}: {count}")

            # 4. Isoelectric point
            pi = analyzed_seq.isoelectric_point()
            print(f"Isoelectric Point (pI): {pi:.2f}")

            # 5. Net charge at physiological pH (7.0)
            charge = analyzed_seq.charge_at_pH(7.0)
            print(f"Net Charge (at pH 7.0): {charge:.2f}")

            # 6. Extinction coefficient
            # Returns a tuple: (reduced Cys, oxidized Cys forming disulfide bridges)
            ext_coeff = analyzed_seq.molar_extinction_coefficient()
            print(f"Extinction Coefficient (reduced / oxidized): {ext_coeff[0]} / {ext_coeff[1]} M^-1 cm
else:
    print("No FASTA data found. Please run the upload and storage cells first.")
```

```
  W: 5
  Y: 14
Isoelectric Point (pI): 4.05
Net Charge (at pH 7.0): -703.49
Extinction Coefficient (reduced / oxidized): 48360 / 48985 M^-1 cm^-1

--- Physiochemical Properties for: g-15451 ---
Length: 335 amino acids
Molecular Weight: 37663.53 Da
Amino Acid Composition:
  A: 33
  C: 8
  D: 12
  E: 19
  F: 15
  G: 30
  H: 8
  I: 11
  K: 15
  L: 18
  M: 8
  N: 24
  P: 11
  Q: 18
  R: 20
  S: 24
  T: 19
  V: 15
  W: 10
  Y: 17
Isoelectric Point (pI): 8.53
Net Charge (at pH 7.0): 4.16
Extinction Coefficient (reduced / oxidized): 80330 / 80830 M^-1 cm^-1
```

Create a Property Dictionary

```
import pandas as pd

protein_properties = []

if 'fasta_data_dict' in locals() and fasta_data_dict:
    for filename, fasta_data in fasta_data_dict.items():
        records = list(SeqIO.parse(io.StringIO(fasta_data), 'fasta'))
        for record in records:
            sequence = str(record.seq).replace('*', '')
            analyzed_seq = ProteinAnalysis(sequence)

            # Create a dictionary for the current protein
            prop_dict = {
                'Protein_ID': record.id,
                'Length': len(sequence),
                'Molecular_Weight_Da': round(analyzed_seq.molecular_weight(), 2),
                'Isoelectric_Point': round(analyzed_seq.isoelectric_point(), 2),
                'Net_Charge_pH7': round(analyzed_seq.charge_at_pH(7.0), 2),
                'Extinction_Coeff_Reduced': analyzed_seq.molar_extinction_coefficient()[0]
            }
            protein_properties.append(prop_dict)

    # Convert the list of dictionaries into a pandas DataFrame
    properties_df = pd.DataFrame(protein_properties)
    print("Protein Properties Summary:")
    display(properties_df)
else:
    print("No FASTA data found to create the dictionary.")
```

Protein Properties Summary:

	Protein_ID	Length	Molecular_Weight_Da	Isoelectric_Point	Net_Charge_pH7	Extinction_Coeff_Reduced
0	g-680	462	52501.82	5.56	-7.98	131670
1	g-11155	335	37421.28	8.14	2.28	67840
2	g-15657	340	38453.17	6.76	-0.74	95800
3	g-4430	333	36689.52	7.17	0.45	64860
4	g-15835	2652	286578.77	4.05	-703.49	48360
5	g-15451	335	37663.53	8.53	4.16	80330

Next steps: Generate code with properties_df New interactive sheet

Sequence Similarity

```
from Bio import Align
from Bio import SeqIO
import io
import itertools

sequences = {}

# Extract all sequences and their IDs
if 'fasta_data_dict' in locals() and fasta_data_dict:
    for filename, fasta_data in fasta_data_dict.items():
        records = list(SeqIO.parse(io.StringIO(fasta_data), 'fasta'))
        for record in records:
            sequence = str(record.seq).replace('*', '')
            sequences[record.id] = sequence

    # Check if we have at least 2 sequences to compare
    if len(sequences) < 2:
        print("Need at least 2 sequences to calculate similarity. Please upload more files.")
    else:
        print(f"Calculating pairwise similarities for {len(sequences)} sequences...\n")

        # Initialize the aligner
        aligner = Align.PairwiseAligner()
        aligner.mode = 'global' # Using global alignment

        # Generate all unique pairs of sequences
        pairs = list(itertools.combinations(sequences.keys(), 2))

        for id1, id2 in pairs:
            seq1 = sequences[id1]
            seq2 = sequences[id2]

            # Perform the alignment
            alignments = aligner.align(seq1, seq2)
            best_alignment = alignments[0]

            # Calculate a basic similarity percentage based on the alignment score and max length
            max_len = max(len(seq1), len(seq2))
            similarity_score = (best_alignment.score / max_len) * 100

            print(f"Alignment between {id1} and {id2}:")
            print(f"  Similarity Score: {similarity_score:.2f}%")
            print(f"  Raw Score: {best_alignment.score}")
            print("-" * 40)
else:
    print("No FASTA data found. Please run the previous storage cells first.")
```

```
  Raw Score: -24.0
----------------------------------------
Alignment between g-680 and g-15657:
  Similarity Score: -4.55%
  Raw Score: -21.0
----------------------------------------
Alignment between g-680 and g-4430:
  Similarity Score: -4.76%
  Raw Score: -22.0
----------------------------------------
Alignment between g-680 and g-15835:
  Similarity Score: -72.29%
  Raw Score: -1917.0
----------------------------------------
Alignment between g-680 and g-15451:
  Similarity Score: -6.49%
  Raw Score: -30.0
----------------------------------------
Alignment between g-11155 and g-15657:
  Similarity Score: 38.53%
  Raw Score: 131.0
----------------------------------------
Alignment between g-11155 and g-4430:
  Similarity Score: 83.58%
  Raw Score: 280.0
----------------------------------------
Alignment between g-11155 and g-15835:
  Similarity Score: -79.34%
  Raw Score: -2104.0
----------------------------------------
Alignment between g-11155 and g-15451:
  Similarity Score: 42.69%
  Raw Score: 143.0
----------------------------------------
Alignment between g-15657 and g-4430:
  Similarity Score: 37.94%
  Raw Score: 129.0
----------------------------------------
Alignment between g-15657 and g-15835:
  Similarity Score: -79.07%
  Raw Score: -2097.0
----------------------------------------
Alignment between g-15657 and g-15451:
  Similarity Score: 71.76%
  Raw Score: 244.0
----------------------------------------
Alignment between g-4430 and g-15835:
  Similarity Score: -79.34%
  Raw Score: -2104.0
----------------------------------------
Alignment between g-4430 and g-15451:
  Similarity Score: 42.09%
  Raw Score: 141.0
----------------------------------------
Alignment between g-15835 and g-15451:
  Similarity Score: -79.37%
  Raw Score: -2105.0
----------------------------------------
```

Store Sequence Similarity

```
import pandas as pd

similarity_results = []

if 'sequences' in locals() and len(sequences) >= 2:
    # Re-initialize the aligner to regenerate the scores for storage
    aligner = Align.PairwiseAligner()
    aligner.mode = 'global'

    pairs = list(itertools.combinations(sequences.keys(), 2))
```

```
    for id1, id2 in pairs:
        seq1 = sequences[id1]
        seq2 = sequences[id2]

        alignments = aligner.align(seq1, seq2)
        best_alignment = alignments[0]

        max_len = max(len(seq1), len(seq2))
        similarity_score = (best_alignment.score / max_len) * 100

        # Append the results to our list
        similarity_results.append({
            'Protein_1': id1,
            'Protein_2': id2,
            'Raw_Score': best_alignment.score,
            'Similarity_Percentage': round(similarity_score, 2)
        })

    # Create a DataFrame from the results
    similarity_df = pd.DataFrame(similarity_results)

    print("Sequence Similarity Summary:")
    display(similarity_df)

    print("\n" + "="*50)
    print("--- Explanation of Similarity Scores ---")
    print("="*50)
    print("1. Raw Score:")
    print("   This is the absolute score calculated by Biopython's PairwiseAligner.")
    print("   Points are added for matching amino acids, and subtracted for mismatches or gaps.")
    print("   A negative score (like -63.0) means the penalty for mismatches and gaps heavily")
    print("   outweighed the positive matches, indicating the sequences are highly divergent.")
    print("\n2. Similarity Percentage:")
    print("   This is a normalized metric created by dividing the Raw Score by the length of")
    print("   the longer sequence. While a true percentage of identity would be between 0-100%,")
    print("   this metric can be negative if the Raw Score is negative.")
else:
    print("Not enough sequences found to store similarity data. Please run the alignment cell first.")
```

```
Sequence Similarity Summary:
```

	Protein_1	Protein_2	Raw_Score	Similarity_Percentage
0	g-680	g-11155	-24.0	-5.19
1	g-680	g-15657	-21.0	-4.55
2	g-680	g-4430	-22.0	-4.76
3	g-680	g-15835	-1917.0	-72.29
4	g-680	g-15451	-30.0	-6.49
5	g-11155	g-15657	131.0	38.53
6	g-11155	g-4430	280.0	83.58
7	g-11155	g-15835	-2104.0	-79.34
8	g-11155	g-15451	143.0	42.69
9	g-15657	g-4430	129.0	37.94
10	g-15657	g-15835	-2097.0	-79.07
11	g-15657	g-15451	244.0	71.76
12	g-4430	g-15835	-2104.0	-79.34
13	g-4430	g-15451	141.0	42.09
14	g-15835	g-15451	-2105.0	-79.37

```
==================================================
--- Explanation of Similarity Scores ---
==================================================
1. Raw Score:
   This is the absolute score calculated by Biopython's PairwiseAligner.
   Points are added for matching amino acids, and subtracted for mismatches or gaps.
   A negative score (like -63.0) means the penalty for mismatches and gaps heavily
   outweighed the positive matches, indicating the sequences are highly divergent.

2. Similarity Percentage:
   This is a normalized metric created by dividing the Raw Score by the length of
   the longer sequence. While a true percentage of identity would be between 0-100%,
   this metric can be negative if the Raw Score is negative.
```

Next steps: Generate code with similarity_df | New interactive sheet

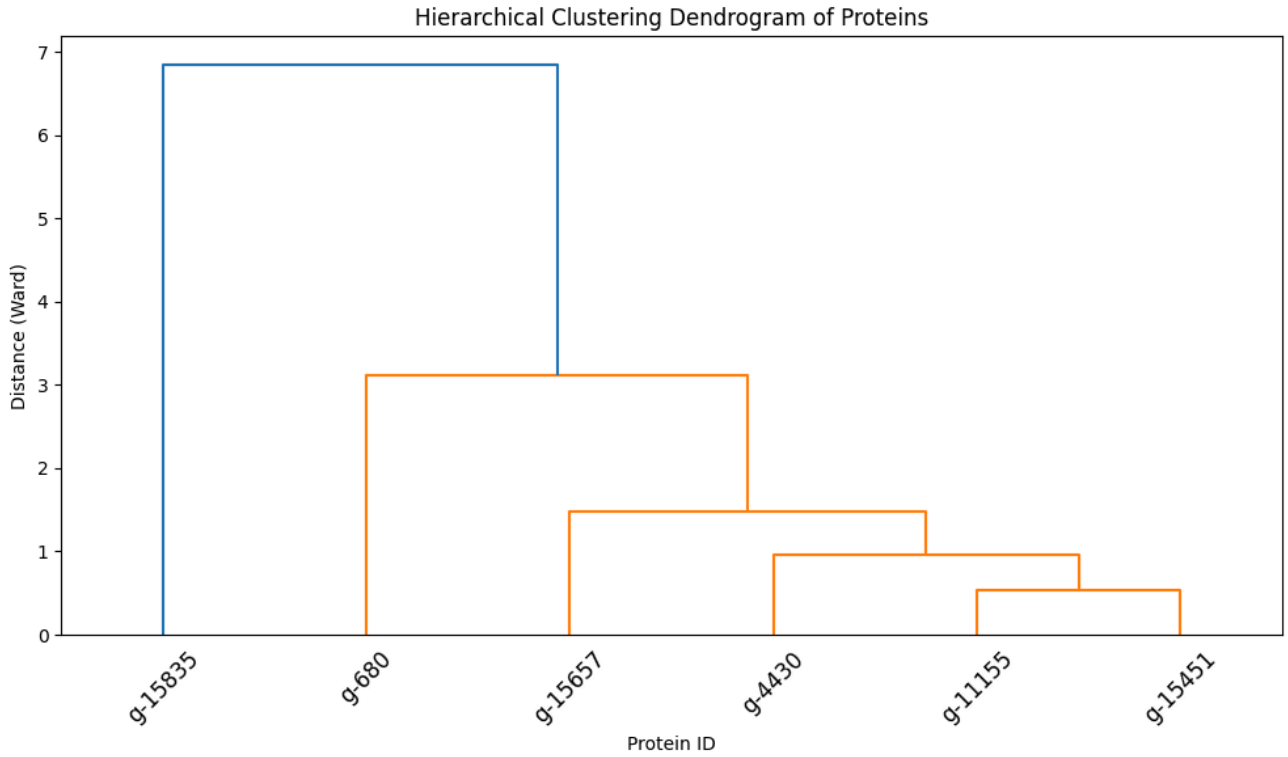
Relationship Between Proteins

Dendogram

```
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler
import numpy as np

if 'properties_df' in locals() and not properties_df.empty:
    if len(properties_df) < 2:
        print("Need at least 2 proteins to generate a dendrogram.")
    else:
        print(f"Generating dendrogram for {len(properties_df)} proteins...\n")

        # Select the numerical features for clustering
        features = ['Length', 'Molecular_Weight_Da', 'Isoelectric_Point', 'Net_Charge_pH7', 'Extinction
        clustering_data = properties_df.set_index('Protein_ID')[features]

        # Standardize the features so each property contributes equally to the distance
        scaler = StandardScaler()
```

```
        scaled_features = scaler.fit_transform(clustering_data)

        # Perform hierarchical clustering using Ward's method
        # Ward minimizes the variance of the clusters being merged
        Z = linkage(scaled_features, method='ward')

        # Plot the dendrogram
        plt.figure(figsize=(10, 6))
        plt.title('Hierarchical Clustering Dendrogram of Proteins')
        plt.xlabel('Protein ID')
        plt.ylabel('Distance (Ward)')

        dendrogram(
            Z,
            labels=clustering_data.index.tolist(),
            leaf_rotation=45.,
            leaf_font_size=12.,
            show_contracted=True
        )

        plt.tight_layout()
        plt.show()

        print("\n--- Dendrogram Explanation ---")
        print("The dendrogram visually represents the relationships between proteins based on")
        print("their physiochemical properties. Proteins that join together at lower heights (y-axis)")
        print("are more similar to each other. We standardized the data so properties with large number
        print("(like Molecular Weight) don't unfairly outweigh properties with small numbers (like Net
else:
    print("Properties data not found. Please run the 'Create a Property Dictionary' cell first.")
```

```
Generating dendrogram for 6 proteins...
```




```
--- Dendrogram Explanation ---
The dendrogram visually represents the relationships between proteins based on
their physiochemical properties. Proteins that join together at lower heights (y-axis)
are more similar to each other. We standardized the data so properties with large numbers
(like Molecular Weight) don't unfairly outweigh properties with small numbers (like Net Charge).
```

Relationship Outputs

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.spatial.distance import pdist, squareform
from sklearn.preprocessing import StandardScaler, MinMaxScaler

if 'properties_df' in locals() and 'similarity_df' in locals() and len(properties_df) >= 2:
    protein_ids = properties_df['Protein_ID'].tolist()

    # 1. Physicochemical Distance Matrix
    features = ['Length', 'Molecular_Weight_Da', 'Isoelectric_Point', 'Net_Charge_pH7', 'Extinction_Coef
    clustering_data = properties_df.set_index('Protein_ID')[features]

    scaler = StandardScaler()
    scaled_features = scaler.fit_transform(clustering_data)

    # Calculate pairwise euclidean distances and convert to square matrix
    phys_dist_array = pdist(scaled_features, metric='euclidean')
    phys_dist_matrix = squareform(phys_dist_array)

    # Normalize to 0-1 for combining later
    mm_scaler = MinMaxScaler()
    phys_dist_norm = mm_scaler.fit_transform(phys_dist_matrix)

    # 2. Sequence Similarity Distance Matrix
    # Initialize with high similarity for identical proteins (diagonal)
    seq_sim_matrix = pd.DataFrame(100.0, index=protein_ids, columns=protein_ids)
    for _, row in similarity_df.iterrows():
        p1, p2, sim = row['Protein_1'], row['Protein_2'], row['Similarity_Percentage']
        seq_sim_matrix.loc[p1, p2] = sim
        seq_sim_matrix.loc[p2, p1] = sim

    # Convert similarity to distance
    max_sim = seq_sim_matrix.values.max()
    seq_dist_matrix = max_sim - seq_sim_matrix.values

    # Normalize to 0-1
    seq_dist_norm = mm_scaler.fit_transform(seq_dist_matrix)

    # 3. Combined Distance Matrix
    combined_dist_norm = (phys_dist_norm + seq_dist_norm) / 2.0

    # Plotting
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    sns.heatmap(phys_dist_norm, xticklabels=protein_ids, yticklabels=protein_ids, ax=axes[0], cmap='YlGn
    axes[0].set_title('1. Physicochemical Distances')

    sns.heatmap(seq_dist_norm, xticklabels=protein_ids, yticklabels=protein_ids, ax=axes[1], cmap='YlGnB
    axes[1].set_title('2. Sequence Similarity Distances')

    sns.heatmap(combined_dist_norm, xticklabels=protein_ids, yticklabels=protein_ids, ax=axes[2], cmap='
    axes[2].set_title('3. Combined Distances')

    plt.tight_layout()
    plt.show()

    print("\n--- Relationship Heatmaps Explanation ---")
    print("These heatmaps show the pairwise 'distances' (dissimilarity) between proteins on a 0 to 1 sca
    print("0 (lighter color) means identical/highly similar, and 1 (darker color) means highly dissimila
    print("1. Physicochemical: Distance based on properties like molecular weight, pI, and charge.")
    print("2. Sequence Similarity: Distance mathematically derived from your alignment scores.")
    print("3. Combined: The average of the physicochemical and sequence distances.")
else:
    print("Required data not found. Please ensure you have run both the property dictionary and sequence
```

1. Physicochemical Distances
g-680
g-11155
0
0.54
0.3
0.53
0.98
0.48
0.53
0
0.26
0.13
0.96
0.094
1.0
0.8
2. Sequence Similarity Distances
g-680
g-11155
0
0.59
0.58
0.58
0.96
0.59
0.61
0
0.34
0.092
1
0.32
1.0
0.8
3. Combined Distances
g-680
g-11155
0
0.56
0.44
0.56
0.97
0.54
0.57
0
0.3
0.11
0.98
0.21
1.0
0.8